Гоцкало Максим 212 группа

Task 5

Командный интерпретатор SHELL

Отчет

2024

Содержание:

- Синтаксис shell
- Тесты
- Структура дерева
- Лексический и синтаксический разборы

Синтаксис shell

Синтаксис команд shell определяется следующими правилами:

⟨команда_shell⟩::= ⟨список_команд⟩

⟨список_команд⟩::= ⟨конвейер⟩ { [один из **&**, **&&**, **||**, **;**] ⟨конвейер⟩} [**&**, **;**]

⟨конвейер⟩::= ⟨команда⟩ { **|** ⟨команда⟩}

⟨команда⟩::= ⟨простая_команда⟩ |

(⟨список_команд⟩**)** [**<**⟨имя_файла⟩] [[один из **>**, **>>**] ⟨имя_файла⟩]

⟨простая_команда>::=⟨имя_файла⟩ {⟨аргумент⟩}

[**<**⟨имя_файла⟩] [[один из **>**, **>>**] ⟨имя_файла⟩]

Аргумент команды — это имя файла, флаг, ключ и т. пр. Конструкция "{...}"означает повторение несколько раз фрагмента между фигурными скобками или его отсутствие. Конструкция "[один из ...]" означает обязательное присутствие одного (и только одного) из перечисленных в ней элементов. Конструкция "[...]" означает возможное присутствие одного из перечисленных в ней через запятую элементов.

Реализованы следующие внутренние команды:

cd – изменяет текущую директорию,

pwd – печатает полный путь к текущей директории

exit – завершает работу интерпретатора.

На этапе лексического анализа обрабатываются следующие символы:

• кавычки (позволяют вставить пробел в аргумент или имя программы), например: cat ”new file” – вывести на экран файл с именем ”new file”

• \ (экранирование символа, отменяет специальное действие последующего символа, например \” трактуется как собственно символ кавычка, а не ограничитель закавыченной подстроки), последовательность \\ позволяет ввести обычный одинарный слеш

• комментарии: при наличии во входной строке символа # (не внутри кавычек, не экранированного обратным слешем) все символы, стоящие после решётки игнорируются.

Также на этапе лексического анализа реализована подстановку четырёх переменных:

• $HOME (домашняя директория пользователя)

• $SHELL (путь к исполняемому в данный момент шеллу)

• $USER (имя пользователя, запустившего процесс)

• $EUID (идентификатор пользователя, с правами которого работает процесс).

Операции >, >> позволяют перенаправить в файл результаты работы любой команды, предназначенные для стандартного вывода.

Через точку с запятой (;) указываются последовательно выполняемые команды (конвейеры).

Символ && (||) означает, что следующая за ним команда будет выполнена только в том случае, если предыдущая команда завершилась успешно (неуспешно).

Операция & указывает, что предшествующая ей команда (конвейер) выполняется в фоновом режиме.

Конвейер (|) позволяет запустить указанные в нем команды в виде отдельных параллельно работающих процессов, соединив каналом стандартный выходной поток каждого процесса (кроме последнего) со стандартным входным потоком следующего процесса.

Круглые скобки означают, что для выполнения указанного в них списка команд создается новый процесс shell (т.н. subshell).

Тесты

Построение дерева:

pwd > f

```
[$=> pwd > f

 argv:
     pwd
 infile = NULL
 outfile = f
 outfile index = 0
 backgrnd = 0
 next exit mode = 0
 psubcmd = NULL
 pipe = NULL
 next = NULL

 $=>
```

sort –r /etc/passwd &

```
$=> sort –r /etc/passwd &

argv:
     sort
     –r
     /etc/passwd
infile = NULL
outfile = NULL
outfile index = 0
backgrnd = 1
next exit mode = 0
psubcmd = NULL
pipe = NULL
next:
     argv = NULL
     infile = NULL
     outfile = NULL
     outfile index = 0
     backgrnd = 0
     next exit mode = 0
     psubcmd = NULL
     pipe = NULL
     next = NULL

$=>
```

```
cat -n <abc >f;
```

```
$=> cat -n <abc >f;

argv:
    cat
    -n
infile = abc
outfile = f
outfile index = 0
backgrnd = 0
next exit mode = 0
psubcmd = NULL
pipe = NULL
next:
    argv = NULL
    infile = NULL
    outfile = NULL
    outfile index = 0
    backgrnd = 0
    next exit mode = 0
    psubcmd = NULL
    pipe = NULL
    next = NULL

$=>
```

ls -l | sort -r | cat –n

```
$=> ls -l | sort -r | cat -n

argv:
    ls
    -l
infile = NULL
outfile = NULL
outfile index = 0
backgrnd = 0
next exit mode = 0
psubcmd = NULL
pipe:
    argv:
        sort
        -r
    infile = NULL
    outfile = NULL
    outfile index = 0
    backgrnd = 0
    next exit mode = 0
    psubcmd = NULL
    pipe:
        argv:
            cat
            -n
        infile = NULL
        outfile = NULL
        outfile index = 0
        backgrnd = 0
        next exit mode = 0
        psubcmd = NULL
        pipe = NULL
        next = NULL
    next = NULL
next = NULL

$=>
```

date; (ls -l|cat -n)>f & pwd

```
$=> date; (ls -l|cat -n)>f & pwd

argv:
    date
infile = NULL
outfile = NULL
outfile index = 0
backgrnd = 0
next exit mode = 0
psubcmd = NULL
pipe = NULL
next:
    argv = NULL
    infile = NULL
    outfile = f
    outfile index = 0
    backgrnd = 1
    next exit mode = 0
    psubcmd:
        argv:
            ls
            -l
        infile = NULL
        outfile = NULL
        outfile index = 0
        backgrnd = 0
        next exit mode = 0
        psubcmd = NULL
        pipe:
            argv:
                cat
                -n
            infile = NULL
            outfile = NULL
            outfile index = 0
            backgrnd = 0
            next exit mode = 0
            psubcmd = NULL
            pipe = NULL
            next = NULL
        next = NULL
    pipe = NULL
    next:
        argv:
            pwd
        infile = NULL
        outfile = NULL
        outfile index = 0
        backgrnd = 0
        next exit mode = 0
        psubcmd = NULL
        pipe = NULL
        next = NULL

$=>
```

Выполнение команд:

pwd;cd;pwd

```
$=> pwd;cd;pwd

/Users/maxbig/2024_gotskalo212/Tasks/task5
/home/maxbig

$=>
```

pwd > fpwd;echo a b c >> fpwd;cat < fpwd > f2pwd;cat < f2pwd



yes | yes | yes | yes | yes | yes | yes | yes | head

```
$=> yes | yes | yes | yes | yes | yes | yes | yes | head

y
y
y
y
y
y
y
y
y
y

$=>
```

ls | cat | cat | cat | cat

```
$=> ls | cat | cat | cat | cat

!
!.c
Makefile
README_5.md
launch_tree.c
launch_tree.o
task3.h
task3_1.c
task3_1.o
task3_f.c
task3_f.o
task5
task5_tree.c
task5_tree.o
test
test.c

$=>
```

pwd > f;yes | yes | yes | yes | sleep 5 | cat < f

pwd | exit | date | cat

sleep 5; pwd

sleep 10 & pwd

sleep 5 & sleep 5 & sleep 5 & sleep 5 & sleep 5 &

echo a b c > f; cat f & ls

((((ls) | cat -n) | cat -n) | sort -r)

pwd && ls && pwd

Pwd || ls

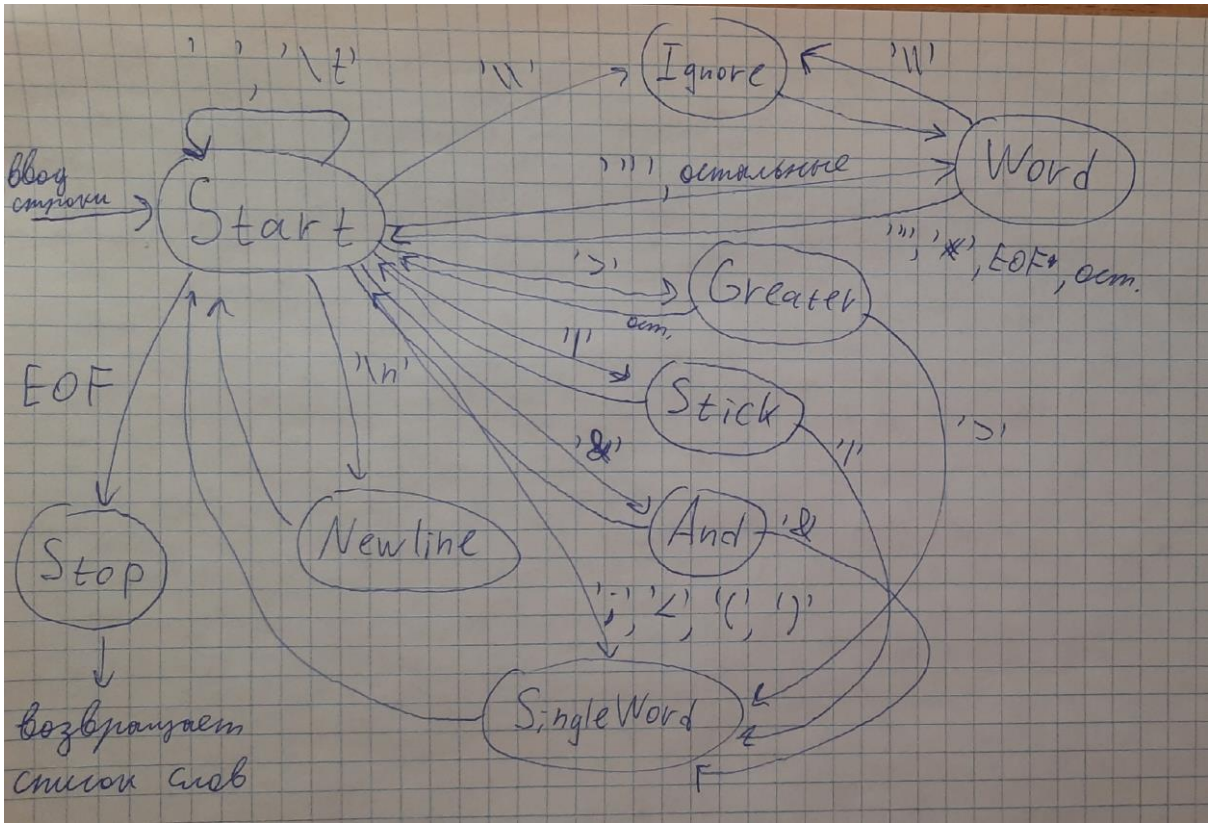
pwd;cd no;pwd;cd;pwd;cd no;pwd

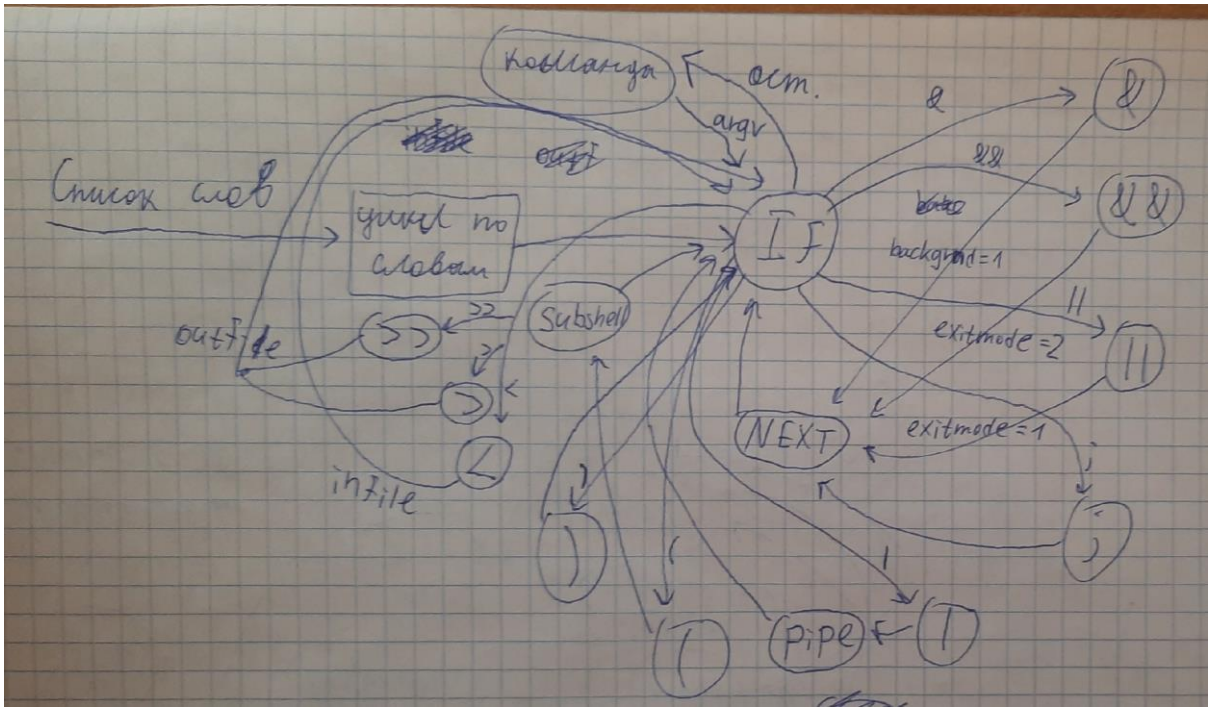
no_command;pwd;no;ls

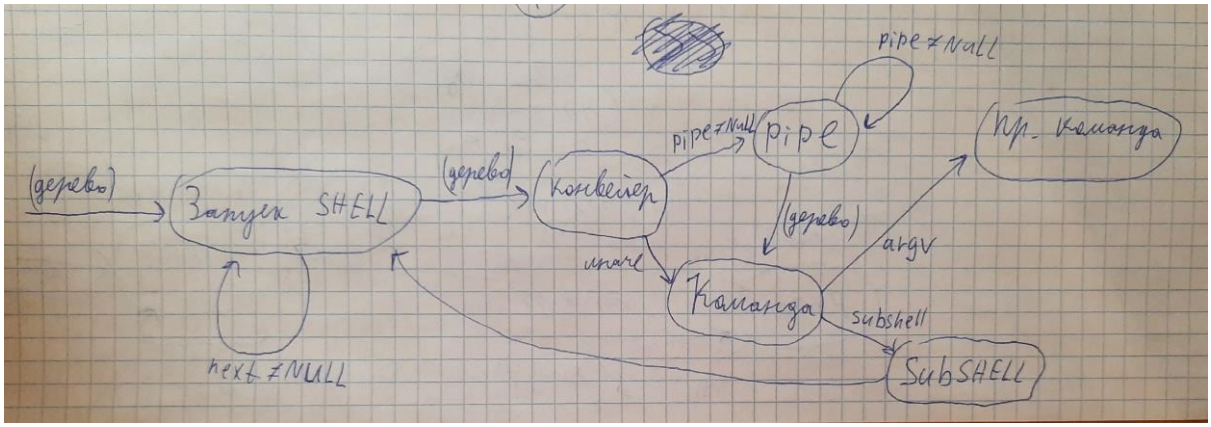
Структура Дерева

```
struct cmd_inf {
    char **argv; // список из имени команды и аргументов
    char *infile; // переназначенный файл стандартного ввода
    char *outfile; // переназначенный файл стандартного вывода
    int outfile_index; // =0, если указан > (запись в начало файла),
                       //=1, если указан >> (запись в конец файла)
    int backgrnd; // =1, если команда подлежит выполнению в фоновом режиме
    int next_exit_mode; // =0, если не было || и &&, =1, если был ||, =2,
                                                        //если был &&
    struct cmd_inf *psubcmd; // команды для запуска в дочернем shell
    struct cmd_inf *pipe; // следующая команда после “|”
    struct cmd_inf *next; // следующая после “;” (или после “&”)
};
```

Лексический и синтаксический разборы


' ', '\t'
'\\'
Ignore
'\''
Word
Ввод строки
Start
'"', остальные
'"', '*', EOF, осм.
'>'
Greater
осм.
'|'
Stick
EOF
'\n'
'&'
'>'
Stop
Newline
And
'&'
';', '<', '(', ')'
SingleWord
Возвращает список слов


Команда
осм.
argv
&
&&
Список слов
цикл по словам
IF
backgrond=1
outfile
>>
Subshell
exitmode=2
||
>
NEXT
exitmode=1
<
infile
;
)
(
|
Pipe
|


pipe≠Null
pipe
пр. команда
(дерево)
Запуск SHELL
(дерево)
Конвейер
pipe≠Null
(дерево)
argv
иначе
Команда
subshell
SubSHELL
next≠NULL